Lessons from Building Claude Code: Seeing like an Agent

Thariq @trq212 · 27/02/2026

https://x.com/trq212/status/2027463795355095314

One of the hardest parts of building an agent harness is constructing its action space.

Claude acts through Tool Calling, but there are a number of ways tools can be constructed in the Claude API with primitives like bash, skills and recently code execution (read more about programmatic tool calling on the Claude API in @RLanceMartin's new article).

Given all these options, how do you design the tools of your agent? Do you need just one tool like code execution or bash? What if you had 50 tools, one for each use case your agent might run into?

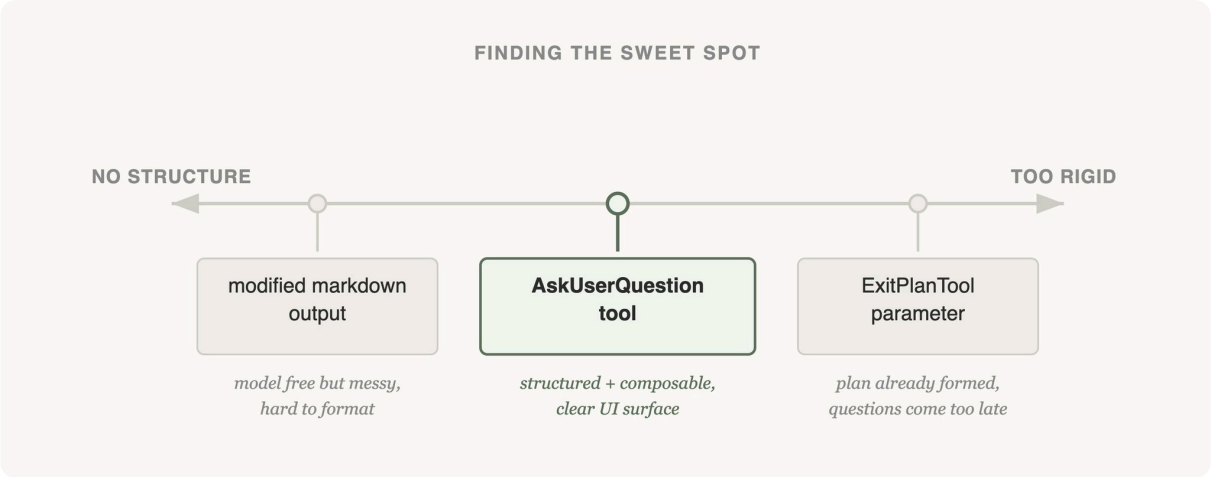
To put myself in the mind of the model I like to imagine being given a difficult math problem. What tools would you want in order to solve it? It would depend on your own skills!

Paper would be the minimum, but you'd be limited by manual calculations. A calculator would be better, but you would need to know how to operate the more advanced options. The fastest and most powerful option would be a computer, but you would have to know how to use it to write and execute code.

This is a useful framework for designing your agent. You want to give it tools that are shaped to its own abilities. But how do you know what those abilities are? You pay attention, read its outputs, experiment. You learn to see like an agent.

Here are some lessons we've learned from paying attention to Claude while building Claude Code.

Improving Elicitation & the AskUserQuestion tool



When building the AskUserQuestion tool, our goal was to improve Claude's ability to ask questions (often called elicitation).

While Claude could just ask questions in plain text, we found answering those questions felt like they took an unnecessary amount of time. How could we lower this friction and increase the bandwidth of communication between the user and Claude?

Attempt #1 - Editing the ExitPlanTool

The first thing we tried was adding a parameter to the ExitPlanTool to have an array of questions alongside the plan. This was the easiest thing to implement, but it confused Claude because we were simultaneously asking for a plan and a set of questions about the plan. What if the user's answers conflicted with what the plan said? Would Claude need to call the ExitPlanTool twice? We needed another approach.

(you can read more about why we made an ExitPlanTool in our post on prompt caching)

Attempt #2 - Changing Output Format

Next we tried modifying Claude's output instructions to serve a slightly modified markdown format that it could use to ask questions. For example, we could ask it to output a list of bullet point questions with alternatives in brackets. We could then parse and format that question as UI for the user.

While this was the most general change we could make and Claude even seemed to be okay at outputting this, it was not guaranteed. Claude would append extra sentences, omit options, or use a different format altogether.

Attempt #3 - The AskUserQuestion Tool

```
← ☐ Paradigm  ☐ Theme  ☐ Snacks  ☐ Indentation  ✓ Submit  →

Which programming paradigm do you prefer for this project?

❯ 1. Functional
     Pure functions, immutability, composition over inheritance
  2. Object-Oriented
     Classes, encapsulation, inheritance hierarchies
  3. Procedural
     Straightforward step-by-step functions and state
  4. Mixed
     Whatever fits the problem best, no strict rules
  5. Type something.
  ──────────────────────────────────────────────────────────────
  6. Chat about this

Enter to select · Tab/Arrow keys to navigate · Esc to cancel
```

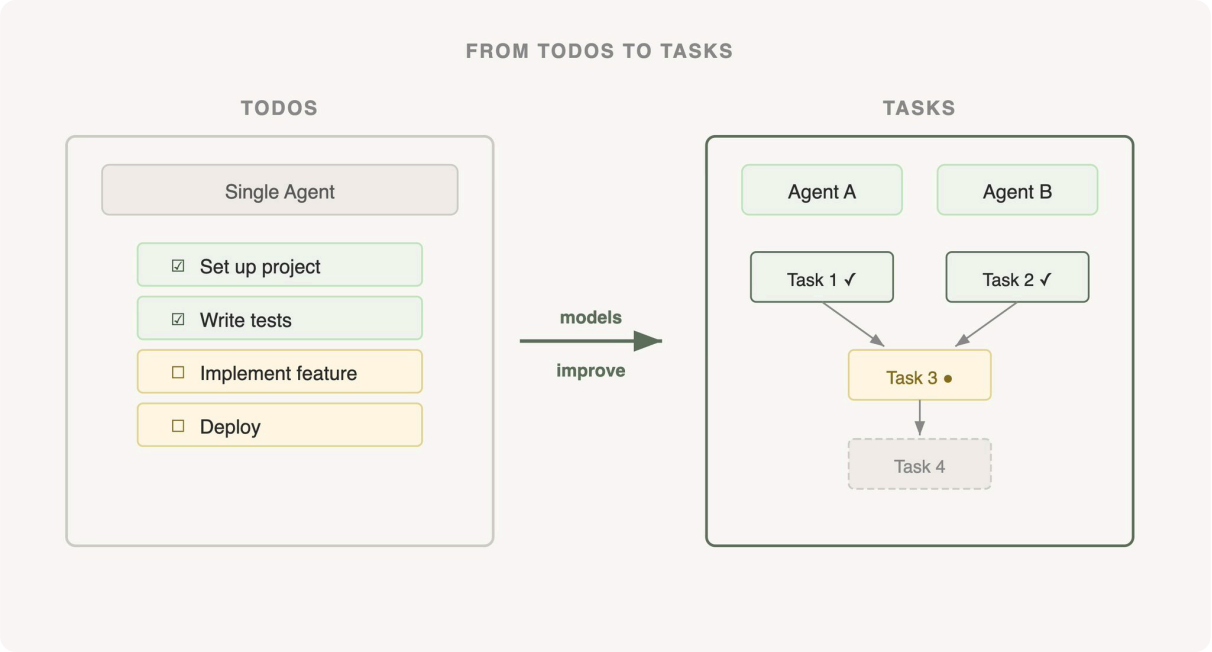
Finally, we landed on creating a tool that Claude could call at any point, but it was particularly prompted to do so during plan mode. When the tool triggered we would show a modal to display the questions and block the agent's loop until the user answered.

This tool allowed us to prompt Claude for a structured output and it helped us ensure that Claude gave the user multiple options. It also gave users ways to compose this functionality, for example calling it in the Agent SDK or using referring to it in skills.

Most importantly, Claude seemed to like calling this tool and we found its outputs worked well. Even the best designed tool doesn’t work if Claude doesn’t understand how to call it.

Is this the final form of elicitation in Claude Code? We’re not sure. As you’ll see in the next example, what works for one model may not be the best for another.

Updating with Capabilities - Tasks & Todos



When we first launched Claude Code, we realized that the model needed a Todo list to keep it on track. Todos could be written at the start and checked off as the model did work. To do this we gave Claude the TodoWrite tool, which would write or update Todos and display them to the user.

But even then we often saw Claude forgetting what it had to do. To adapt, we inserted system reminders every 5 turns that reminded Claude of its goal.

But as models improved, they not only did not need to be reminded of the Todo List but could find it limiting. Being sent reminders of the todo list made Claude think that it had to stick to the list instead of modifying it. We also saw Opus 4.5 also get much better at using subagents, but how could subagents coordinate on a shared Todo List?

Seeing this, we replaced TodoWrite with the Task Tool (read more on Tasks here). Whereas Todos were about keeping the model on track, Tasks were more about helping agents communicate with each other. Tasks could include dependencies, share updates across subagents and the model could alter and delete them.

As model capabilities increase, the tools that your models once needed might now be constraining them. It’s important to constantly revisit previous assumptions on what tools are needed. This is also why it's useful to stick to a small set of models to support that have a fairly similar capabilities profile.

Designing a Search Interface

A particularly important set of tools for Claude are the search tools that can be used to build its own context.

When Claude Code first came out, we used a RAG vector database to find context for Claude. While RAG was powerful and fast it required indexing and setup and could be fragile across a host of different environments. More importantly, Claude was *given* this context instead of finding the context itself.

But if Claude could search on the web, why not search your codebase? By giving Claude a Grep tool, we could let it search for files and build context itself.

This is a pattern we've seen as Claude gets smarter, it becomes increasingly good at building its context if it's given the right tools.

When we introduced Agent Skills we formalized the idea of progressive disclosure, which allows agents to incrementally discover relevant context through exploration.

Claude could read skill files and those files could then reference other files that the model could read recursively. In fact, a common use of skills is to add more search capabilities to Claude like giving it instructions on how to use an API or query a database.

Over the course of a year Claude went from not really being able to build its own context, to being able to do nested search across several layers of files to find the exact context it needed.

Progressive disclosure is now a common technique we use to add new functionality without adding a tool.

Progressive Disclosure - The Claude Code Guide Agent

Claude Code currently has ~20 tools, and we are constantly asking ourselves if we need all of them. The bar to add a new tool is high, because this gives the model one more option to think about.

For example, we noticed that Claude did not know enough about how to use Claude Code. If you asked it how to add a MCP or what a slash command did, it would not be able to reply.

We could have put all of this information in the system prompt, but given that users rarely asked about this, it would have added context rot and interfered with Claude Code’s main job: writing code.

Instead, we tried a form of progressive disclosure. We gave Claude a link to its docs which it could then load to search for more information. This worked but we found that Claude would load a lot of results into context to find the right answer when really all you needed was the answer.

So we built the Claude Code Guide subagent which Claude is prompted to call when you ask about itself, the subagent has extensive instructions on how to search docs well and what to return.

While this isn’t perfect, Claude can still get confused when you ask it about how to set itself up, it is much better than it used to be! We were able to add things to Claude's action space without adding a tool.

An Art, not a Science

If you were hoping for a set of rigid rules on how to build your tools, unfortunately that is not this guide. Designing the tools for your models is as much an art as it is a science. It depends heavily on the model you're using, the goal of the agent and the environment it’s operating in.

Experiment often, read your outputs, try new things. See like an agent.